The Feedback Vertex Set Problem in Undirected Graphs

A 2-Approximation Primal-Dual Algorithm

Mushfiqur Rahman
Student ID: 2005107

February 8, 2026

Outline

1. Problem Definition
2. Key Insight: Edge Counting
3. Mathematical Foundation
4. New Integer Programming Formulation
5. Primal-Dual Algorithm
6. Performance Analysis
7. Proof of Lemma 14.7

The Feedback Vertex Set Problem

Problem Statement

Given an undirected graph $G = (V, E)$ with nonnegative vertex weights $w_v \geq 0$ for $v \in V$:

Goal: Find a minimum-cost subset $S \subseteq V$ such that every cycle C in the graph contains at least one vertex from S.

The Feedback Vertex Set Problem

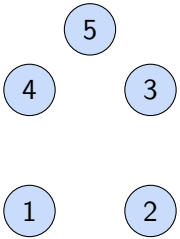
Problem Statement

Given an undirected graph $G = (V, E)$ with nonnegative vertex weights $w_v \geq 0$ for $v \in V$:

Goal: Find a minimum-cost subset $S \subseteq V$ such that every cycle C in the graph contains at least one vertex from S.

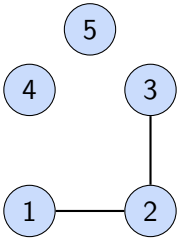
Equivalently: Find a minimum-weight set S such that the induced graph $G(V - S)$ is acyclic.

Visual Example: Building the Graph



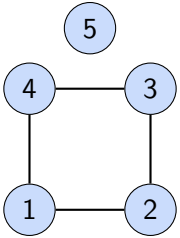
Step 1: The vertices

Visual Example: Building the Graph



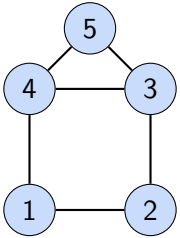
Step 2: Adding edges

Visual Example: Building the Graph



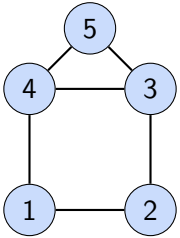
Step 3: Cycle formed! (1-2-3-4-1)

Visual Example: Building the Graph



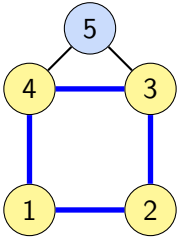
Step 4: Complete graph

Identifying Cycles in the Graph



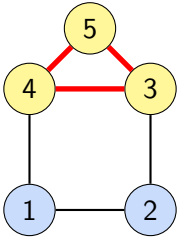
Looking for cycles...

Identifying Cycles in the Graph



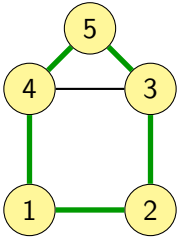
Cycle 1: $1 \to 2 \to 3 \to 4 \to 1$

Identifying Cycles in the Graph



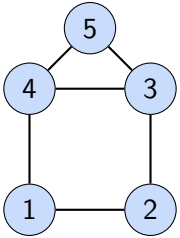
Cycle 2: $3 \to 4 \to 5 \to 3$

Identifying Cycles in the Graph



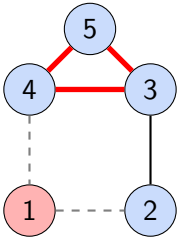
Cycle 3: $1 \to 2 \to 3 \to 5 \to 4 \to 1$

Identifying Cycles in the Graph



Three cycles found! Which vertex(es) to remove?

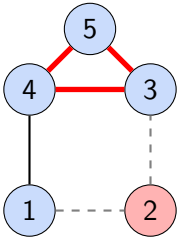
Exploring Different Solutions



Option 1: Remove vertex 1
Result: Still has cycle 3-4-5-3!

Removing vertices 1 or 2 alone doesn't work!

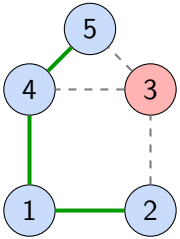
Exploring Different Solutions



Option 2: Remove vertex 2
Result: Still has cycle 3-4-5-3!

Removing vertices 1 or 2 alone doesn't work!

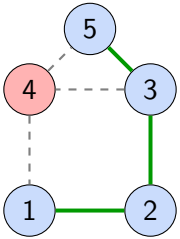
Exploring Different Solutions



Option 3: Remove vertex 3
Result: Graph is acyclic!

Only vertices 3 or 4 break ALL cycles!

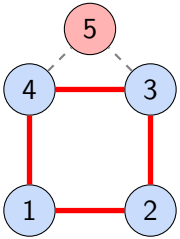
Exploring Different Solutions



Option 4: Remove vertex 4
Result: Graph is acyclic!

Only vertices 3 or 4 break ALL cycles!

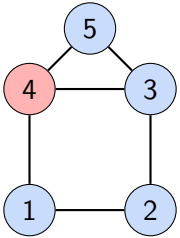
Exploring Different Solutions



Option 5: Remove vertex 5
Result: Still has cycle 1-2-3-4-1!

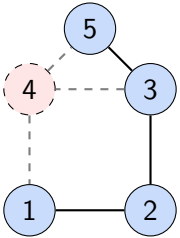
Removing vertex 5 also fails!

The Solution: Removing Vertex 4



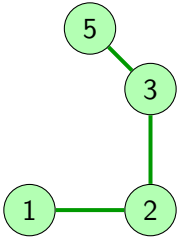
Before: Vertex 4 selected for removal
(Vertex 4 appears in all 3 cycles)

The Solution: Removing Vertex 4



Removing vertex 4 and its edges...
This breaks all 3 cycles!

The Solution: Removing Vertex 4



After: Graph is now acyclic !

Feedback Vertex Set: $S = \{4\}$

Previous Approach

Earlier O(log n)-Approximation (Section 7.2)

Integer Programming Formulation:

$$\begin{aligned} \text{minimize} \quad & \sum_{v \in V} w_v x_v \\ \text{subject to} \quad & \sum_{v \in C} x_v \geq 1, \quad \forall C \in \mathcal{C} \\ & x_v \in \{0, 1\}, \quad \forall v \in V \end{aligned}$$

where $\mathcal{C}$ is the set of all cycles.

Limitation

Integrality gap is $\Theta(\log n)$, cannot achieve better than $O(\log n)$ approximation.

Previous Approach: Parameter Definitions

Parameters in the Formulation

- $G = (V, E)$ = Undirected graph
- V = Set of all vertices in the graph
- $w_v \geq 0$ = Nonnegative weight (cost) of vertex v
- $\mathcal{C}$ = Set of **all cycles** in the graph
- $C \in \mathcal{C}$ = A specific cycle (a subset of vertices forming a cycle)
- $x_v \in \{0, 1\}$ = Decision variable for vertex v
  - $x_v = 1$ if vertex v is selected in the feedback vertex set
  - $x_v = 0$ if vertex v is not selected

Previous Approach: Understanding the Formulation

Objective Function:

$$\text{minimize} \quad \sum_{v \in V} w_v x_v$$

- Minimize the **total weight** of selected vertices
- Only vertices with $x_v = 1$ contribute to the cost

Cycle Constraints:

$$\sum_{v \in C} x_v \geq 1, \quad \forall C \in \mathcal{C}$$

- For **every cycle** C, at least one vertex must be selected
- Ensures every cycle is **broken**
- Need **minimum one constraint per cycle**

Counting Edges in Acyclic Graphs

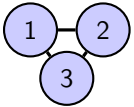
Key Observation

An acyclic subset of edges in graph $G = (V, E)$ can have at most $|V| - c(G)$ edges, where $c(G)$ is the number of connected components.

Therefore, to make the graph acyclic, we must remove at least:

$$\boxed{|E| - |V| + c(G) \text{ edges}}$$

Example:



$|V| = 3, |E| = 3, c(G) = 1$

Remove: $3 - 3 + 1 = 1$ edge

Question: When we remove vertex v of degree $d(v)$, how many edges towards this goal do we effectively remove?

Answer: Not simply $d(v)$! We also reduce vertices to span.

Impact of Removing a Vertex

Scenario: Remove vertex v with degree $d(v)$

Before removal: Need to remove $|E| - |V| + c(G)$ edges

After removal: Need to remove $(|E| - d(v)) - (|V| - 1) + c(G - v)$ edges

Net Reduction

The total number of edges we need to remove decreases by:

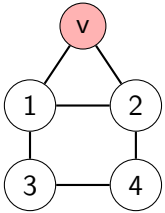
$$\begin{aligned}&[|E| - |V| + c(G)] - [(|E| - d(v)) - (|V| - 1) + c(G - v)]\\&= d(v) - 1 + (c(G) - c(G - v))\end{aligned}$$

Define: $\boxed{b(v) = c(G - v) - c(G) + 1}$ (change in connected components)

Then removing v reduces the edge count by: $d(v) - b(v)$

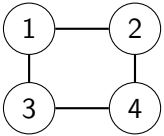
Impact of Removing a Vertex: Example

Original Graph:



$|V| = 5, |E| = 6, c(G) = 1$
$d(v) = 2$

After removing v:



$|V| = 4, |E| = 4, c(G - v) = 1$
$b(v) = 1 - 1 + 1 = 1$

Calculation

Before: $|E| - |V| + c(G) = 6 - 5 + 1 = 2$ edges to remove
After: $(|E| - d(v)) - (|V| - 1) + c(G - v) = 4 - 4 + 1 = 1$ edge to remove
Net reduction: $2 - 1 = 1 = d(v) - b(v) = 2 - 1$ ✓

Lemma 14.4

Theorem (Main Inequality)

For any feedback vertex set F of graph $G = (V, E)$:

$$\sum_{v \in F} (d(v) - b(v)) \geq |E| - |V| + c(G)$$

Proof Outline

Derive two inequalities and add them together.

First Inequality (Edge Counting):

Let $E(F)$ = edges with both endpoints in F. After removing F, remaining edges are acyclic.

- Edges removed: $\sum_{v \in F} d(v) - |E(F)|$
- Edges remaining (acyclic): at most $|V| - |F| - c(G - F)$

Therefore:

$$|E| \leq \sum_{v \in F} d(v) - |E(F)| + |V| - |F| - c(G - F)$$

Lemma 14.4: Completing the Proof

Second Inequality (proven by induction):

$$c(G) \leq \sum_{v \in F} (1 - b(v)) + |E(F)| + c(G - F) \tag{14.2}$$

Adding the two inequalities:

$$\begin{aligned} |E| + c(G) &\leq \sum_{v \in F} d(v) - \cancel{|E(F)|} + |V| - |F| - \cancel{c(G - F)} \\ &\quad + \sum_{v \in F} (1 - b(v)) + \cancel{|E(F)|} + \cancel{c(G - F)} \\ &= \sum_{v \in F} (d(v) - b(v)) + |V| \end{aligned}$$

Subtracting $|V|$ from both sides gives:

$$\boxed{\sum_{v \in F} (d(v) - b(v)) \geq |E| - |V| + c(G)}$$

Proving Inequality (14.2) by Induction

Claim: $c(G) \leq \sum_{v \in F}(1 - b(v)) + |E(F)| + c(G - F)$

Base Cases:

- $F = \emptyset$: $c(G) \leq 0 + 0 + c(G)$ ✓
- $F = \{v\}$: By definition of $b(v)$,

$$c(G) \leq (1 - b(v)) + c(G - v) = (1 - (c(G - v) - c(G) + 1)) + c(G - v) = c(G)$$

✓

Inductive Step: Assume holds for $F - v$, show for F.
When we remove v, its neighbors partition into at least $c(G - v) - c(G) = b(v) - 1$ additional components.
Choose one neighbor of v in each of these $b(v) - 1$ components. For each:

- Either already in F (edge to v in $E(F)$), or
- Not in F (removing v from $G - (F - v)$ creates new component)

Proving Inequality (14.2): Inductive Step

From the observation:

$$b(v) - 1 \leq [|E(F)| - |E(F - v)|] + [c(G - F) - c(G - (F - v))]$$

Rearranging:

$$0 \leq 1 - b(v) + [|E(F)| - |E(F - v)|] + [c(G - F) - c(G - (F - v))]$$

By induction hypothesis:

$$c(G) \leq \sum_{u \in F - v} (1 - b(u)) + |E(F - v)| + c(G - (F - v))$$

Adding these:

$$\begin{aligned} c(G) &\leq \sum_{u \in F - v} (1 - b(u)) + (1 - b(v)) + |E(F)| + c(G - F) \\ &= \sum_{v \in F} (1 - b(v)) + |E(F)| + c(G - F) \quad \checkmark \end{aligned}$$

Corollary 14.5

Corollary

For any subset $S \subseteq V$, let:

- $G(S)$ = *subgraph induced by* S
- $d_S(v)$ = *degree of* v *in* $G(S)$
- $b_S(v)$ = *value of* $b(v)$ *in* $G(S)$
- $f(S) = |E(S)| - |S| + c(G(S))$

Then:

$$\sum_{v \in F \cap S} (d_S(v) - b_S(v)) \geq f(S)$$

Proof: $F \cap S$ is a feedback vertex set for $G(S)$, so Lemma 14.4 applies.

New Integer Program

IP Formulation

$$\begin{aligned} \text{minimize} \quad & \sum_{v \in V} w_v x_v \\ \text{subject to} \quad & \sum_{v \in S} (d_S(v) - b_S(v)) x_v \geq f(S), \quad \forall S \subseteq V \\ & x_v \in \{0, 1\}, \quad \forall v \in V \end{aligned}$$

Why does this work?

- If x is a feedback vertex set: constraints satisfied by Corollary 14.5
- If x is not a feedback vertex set: $\exists$ cycle C with $x_v = 0, \forall v \in C$
  - LHS of constraint for $S = C$ is 0
  - But $f(C) = |E(C)| - |C| + c(G(C)) \geq c(G(C)) \geq 1$
  - Constraint violated!

Linear Programming Dual

The dual of the LP relaxation is:

Dual Program

$$\begin{aligned} \text{maximize} \quad & \sum_{S \subseteq V} f(S) y_S \\ \text{subject to} \quad & \sum_{S : v \in S} (d_S(v) - b_S(v)) y_S \leq w_v, \quad \forall v \in V \\ & y_S \geq 0, \quad \forall S \subseteq V \end{aligned}$$

This dual formulation is the basis for our primal-dual algorithm!

Algorithm 14.2: Overview

$y \leftarrow 0$, $F \leftarrow \emptyset$, $S \leftarrow V$, $\ell \leftarrow 0$
while F is not feasible (i.e., $G(S)$ contains a cycle) **do**
  $\ell \leftarrow \ell + 1$
  Increase y_S until $\sum_{C: v \in C}(d_C(v) - b_C(v))y_C = w_v$ for some $v \in S$
  $F \leftarrow F \cup \{v\}$
  $T \leftarrow \{v \in S : v \text{ not on some cycle in } G(S - v)\}$
  $S \leftarrow S - \{v\} - T$
end while
Clean-up phase:
for $k = \ell$ down to 1 **do**
  if $F - v_k$ is a feasible solution **then**
    Remove v_k from F
  end if
end for
return F

Algorithm Intuition

- **Start:** $y = 0$ (dual feasible), $F = \emptyset$ (primal infeasible)
- **Iteration:** Consider current subgraph $G(S)$
  - Increase y_S until some dual constraint becomes tight for vertex v
  - Add v to F
  - Remove v and vertices no longer in cycles from S
- **Clean-up:** Remove vertices from F in reverse order if they're not needed
  - Ensures F is minimal
  - Critical for the proof!

y_S increases until constraint tight

Lemma 14.6: Minimality

Lemma

For the primal solution F and dual solution y returned by Algorithm 14.2:
If $y_S > 0$, then $F \cap S$ is a minimal feedback vertex set for $G(S)$.

Proof by Contradiction:

- Suppose $\exists v_j \in F \cap S$ such that $F \cap S - v_j$ is a FVS for $G(S)$
- By clean-up: $\{v_1, \ldots, v_{j-1}\} \cup F - v_j$ is not a FVS for G
- Therefore: $\exists$ cycle C in G where v_j is the only vertex from $\{v_1, \ldots, v_{j-1}\} \cup F$
- Since $v_j \in S$ and $\{v_1, \ldots, v_{j-1}\} \cap C = \emptyset$, we have $C \subseteq S$
- But then C is in $G(S)$ and $C \cap F = \{v_j\}$
- Contradiction! $F \cap S - v_j$ cannot be a FVS for $G(S)$

Combinatorial Lemma

The key to the analysis is:

Lemma (Lemma 14.7)

For any graph G where every vertex is in some cycle, and for any minimal feedback vertex set F:

$$\sum_{v \in F} (d(v) - b(v)) \leq 2f(V) = 2(|E| - |V| + c(G))$$

Why is this powerful?

- It bounds the "cost" of the minimal FVS in terms of $d(v) - b(v)$
- Factor of 2 directly gives us the approximation ratio
- Applies to any minimal FVS, not just the algorithm's output

Main Theorem

Theorem (Theorem 14.8)

Algorithm 14.2 is a 2-approximation algorithm for the feedback vertex set problem in undirected graphs.

Proof:

$$\begin{aligned}
\sum_{v \in F} w_v &= \sum_{v \in F} \sum_{S : v \in S} (d_S(v) - b_S(v)) y_S \quad \text{(tight constraints)} \\
&= \sum_{S \subseteq V} y_S \sum_{v \in F \cap S} (d_S(v) - b_S(v)) \\
&\leq \sum_{S \subseteq V} y_S \cdot 2f(S) \quad \text{(by Lemma 14.7)} \\
&= 2 \sum_{S \subseteq V} f(S) y_S \\
&\leq 2 \cdot \mathrm{OPT} \quad \text{(weak duality)}
\end{aligned}$$

Proof Setup

Goal: Show $\sum_{v \in F}(d(v) - b(v)) \leq 2(|E| - |V| + c(G))$

Strategy:

1. Since $\sum_{v \in V} d(v) = 2|E|$, subtract from both sides:

$$\sum_{v \in F} d(v) - \sum_{v \in V} d(v) - \sum_{v \in F} b(v) \leq 2(c(G) - |V|)$$

2. Rearrange to get:

$$\sum_{v \notin F} d(v) \geq 2|V| - \sum_{v \in F} b(v) - 2c(G)$$

3. Break down $\sum_{v \notin F} d(v)$:

$$\sum_{v \notin F} d(v) = \sum_{v \notin F} d_{V-F}(v) + |\delta(F)|$$

where $\delta(F) =$ edges between F and $V - F$

Proof: Forest Property

Since $G(V - F)$ is a forest:

$$\sum_{v \notin F} d_{V-F}(v) = 2(|V| - |F| - c(G - F))$$

Substituting:

$$\begin{aligned} & 2(|V| - |F| - c(G - F)) + |\delta(F)| \geq 2|V| - \sum_{v \in F} b(v) - 2c(G) \\ \Rightarrow \quad & 2|F| + 2c(G - F) \leq |\delta(F)| + \sum_{v \in F} b(v) + 2c(G) \end{aligned}$$

Now we need to show this inequality holds using a charging argument!

Charging Scheme: Key Observations

For minimal FVS F:

- For each $v \in F$, $\exists$ cycle C_v where v is the only vertex of F in C_v
- Therefore, v has two cycle edges from F to $V - F$
- These two edges go to the same connected component of $G(V - F)$

Charging Strategy:

- Charge the $2|F|$ term to the cycle edges in $|\delta(F)|$
- For each component of $G(V - F)$, charge 2 units against:
  - Noncycle edges in $|\delta(F)|$, or
  - Components in $c(G)$, or
  - $b(v)$ values

Case Analysis for Each Component S

For each connected component S of $G(V - F)$:

Case 1: At least 2 noncycle edges from F to S

- Charge 2 to $|\delta(F)|$

Case 2: Exactly 2 edges (both cycle edges from C_v)

- If $v \cup S$ is a component of G: charge 2 to $c(G)$
- Otherwise, removing v disconnects S: $b(v) \geq 2$, charge 2 to $b(v)$

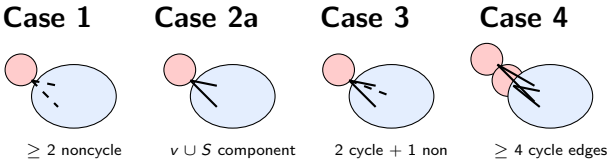
Case 3: 3 edges (2 cycle from C_v, 1 noncycle)

- Charge 1 to noncycle edge, 1 to $b(v) \geq 1$

Case 4: ≥ 4 cycle edges (from C_v and C_w)

- Charge 1 to $b(v) \geq 1$, 1 to $b(w) \geq 1$

Case Analysis: Visual Illustration



Solid lines = cycle edges, Dashed lines = noncycle edges
Red circles = vertices in F, Blue = components of $G(V - F)$

Each case successfully charges 2 units! ✓

Thank You!

Questions?